Privacy-Preserving & Trustworthy Cyber-Infrastructures

CIS 4212/6214 (Spring 2026)

Instructor: **Dr. Attila A. Yavuz**
TA: ***Saif E. Nouma***

Homework 5 (Due: April 1st, 2026 - 23 : 59) [100 Points]

(Upload your solutions to the CANVAS)
Requirements: HW groups are 2-3 people. Each student should contribute all parts of the answers. If one group member drops, the others are equally responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner (if you are doing in group of 2), see syllabus for breaking-up with pair programming partner). Directly borrowing (e.g., copy-paste) from any material and putting in solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations. All your solutions will be passed through MOSS and Turnitin for plagiarism check.**
Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also make research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially be not sufficient.
Remark: HW should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.
As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

Programming Assignment 1 : Arazi-Qi [40 points]

Implementation Setting

- You will use openssl for all cryptogrpahic implementations.
- The files **RequiredFunctions.[h,c]** contains helper functions that you are allowed to use in your implementation. ***Assume these files exists on the testing directory of the instructor's machine.*** You don't need to upload them in your submission. DO NOT MODIFY THESE FILES.
- All cryptographic keys, ciphertexts, and signatures exchanged via files will be Hex-encoded strings.
- You will also be provided with templates for this assignment. They also have detailed instructions on each step you should perform.
- Test Vectors will be provided to validate your outputs.

- A Verification script **VerifyArazi Qi.sh** will be provided to you. use the following command to run it:

```
bash VerifyArazi Qi.sh
```

  Your code, test vectors, required function file and this verification script must be kept in the same directory. This script will run the programs in the following order:

$$CA \rightarrow Alice \rightarrow Bob \rightarrow Alice$$

  It will compare your results with the test vectors by comparing the file names and its contents. So make sure that files generated by your code has correct names and format.

- The following section describes an overview of the Arazi-Qi implementation in this assignment.

Elliptic Curve Based Arazi-Qi

System Parameters

Let $G = \langle P \rangle$ be an elliptic curve group of prime order q. The trusted authority (CA) selects a master secret

$$d \xleftarrow{\$} \mathbb{Z}_q$$

and publishes the master public key

$$D = dP$$

along with hash function $H : \{0,1\}^* \rightarrow \mathbb{Z}_q$.

AQ.Offline (Performed by CA)

For each user A with identity ID_a:

1. Select $b_a \xleftarrow{\$} \mathbb{Z}_q$.
2. Compute public identity component
$$U_a = b_a P.$$
3. Compute private key
$$x_a = H(ID_a \parallel U_a) \cdot b_a + d \pmod q.$$
4. Output (x_a, U_a) to user A.

The same process is repeated for user B.

AQ.Online (Key Exchange Phase)

Node A	**Node B**
$p_a \xleftarrow{\$} \mathbb{Z}_q$	$p_b \xleftarrow{\$} \mathbb{Z}_q$
$E_a = p_a P$	$E_b = p_b P$
$\xrightarrow{(ID_a, U_a, E_a)}$	
$\xleftarrow{(ID_b, U_b, E_b)}$	

Key Computation

Node A computes:

$$K_{ab} = x_a \left[H(ID_b \parallel U_b) \cdot U_b + D \right] + p_a E_b$$

Node B computes:

$$K_{ab} = x_b \left[H(ID_a \parallel U_a) \cdot U_a + D \right] + p_b E_a$$

Correctness

Both parties derive the same shared key:

$$K_{ab} = (x_a x_b + p_a p_b) P$$

1. ca.c

This file implements the **offline phase** of the ECC-based Arazi–Qi identity-based authenticated key exchange protocol. The Certificate Authority (CA) generates long-term, identity-bound keys for Alice and Bob.

Global Parameters and Identities

- Define user identity strings:
  - `ID_A = "alice@example.com"`
  - `ID_B = "bob@example.com"`
- The elliptic curve group is initialized using `init_group(&group, &q)`, where:
  - `group` is the elliptic curve group
  - `q` is the prime group order
- Obtain the generator point `P` using `EC_GROUP_get0_generator(group)`.

Master Key Setup

- Allocate the master secret scalar `d` and master public point `D`.
- Load the master secret `d` from a hex-encoded input file `argv[1]` using `read_bn_hex`.
- Compute the master public key:

  $$D = d \cdot P$$

  using `EC_POINT_mul`.
- Write the master keys to output files:
  - `ca_master_secret_d.txt`
  - `ca_master_public_D.txt`

User-Specific Offline Key Generation

For each user, the CA derives identity-bound private and public components.

Input Scalars

- Load user-specific secret scalars:
  - `b_a` from `argv[2]` (Alice)
  - `b_b` from `argv[3]` (Bob)
- Scalars are read in hex format using `read_bn_hex`.

Alice Key Generation

- Compute Alice's public identity component:

$$U_a = b_a \cdot P$$

  using `EC_POINT_mul`.
- Serialize `U_a` into bytes using `point_to_bytes`.
- Concatenate `ID_A || U_a` into a buffer `buf`.
- Hash the buffer using `sha256_to_scalar` to obtain:

$$h_a = H(\texttt{ID_A}||U_a) \bmod q$$

- Compute Alice's private key:

$$x_a = (h_a \cdot b_a + d) \bmod q$$

  using `BN_mod_mul` and `BN_mod_add`.
- Write Alice's keys to files:
  - `alice_private_xa.txt`
  - `alice_public_Ua.txt`

Bob Key Generation

- Compute Bob's public identity component:

$$U_b = b_b \cdot P$$

  using `EC_POINT_mul`.
- Serialize `U_b` and concatenate with `ID_B`.
- Hash the concatenation to obtain:

$$h_b = H(\texttt{ID_B}||U_b) \bmod q$$

  using `sha256_to_scalar`.
- Compute Bob's private key:

$$x_b = (h_b \cdot b_b + d) \bmod q$$

- Write Bob's keys to files:
  - `bob_private_xb.txt`
  - `bob_public_Ub.txt`

Security Properties

- The master secret `d` never leaves the CA.
- Each user's private key is bound to their identity string.
- The following identity-binding invariant holds for each user U:

$$x_U \cdot P = H(\texttt{ID_U}||U_U) \cdot U_U + D$$

- No online communication or key exchange is performed in this file.

2. alice.c

This file implements **Alice's online phase** of the ECC-based Arazi–Qi identity-based authenticated key exchange protocol. Alice generates an ephemeral Diffie–Hellman component and computes the shared session key.

Inputs and Identity

- Alice's fixed identity string:

```
ID_A = "alice@example.com"
```

- Bob's identity string (used for hashing):

```
ID_B = "bob@example.com"
```

- Input files (hex encoded):
  - `alice_private_xa.txt`
  - `alice_public_Ua.txt`
  - `bob_public_Ub.txt`
  - `ca_master_public_D.txt`

Elliptic Curve Initialization

- Initialize the elliptic curve group and prime order using:

```
init_group(&group, &q)
```

- Obtain the generator point:

$$P = \texttt{EC_GROUP_get0_generator(group)}$$

- Allocate a big-number context `ctx` using `BN_CTX_new`.

Loading Long-Term Parameters

- Load Alice's long-term private scalar `x_a` from `argv[1]` using `read_bn_hex`.
- Load Alice's public identity point `U_a` from `argv[2]` using `read_point_hex`.
- Load Bob's public identity point `U_b` from `argv[4]`.
- Load the CA master public key `D` from `argv[5]`.

Ephemeral Key Generation

- Load Alice's ephemeral scalar `p_a` from `argv[3]`.
- Compute Alice's ephemeral public point:
$$E_a = p_a \cdot P$$
using `EC_POINT_mul`.
- Write the ephemeral public point to:
`alice_ephemeral_Ea.txt`
- Alice conceptually sends the tuple:
$$(\texttt{ID_A}, U_a, E_a)$$
to Bob.

Receiving Bob's Ephemeral Contribution

- Attempt to load Bob's ephemeral public point `E_b` from:
`bob_ephemeral_Eb.txt`
- If the file does not exist, Alice terminates after writing `E_a`.

Hash Computation for Bob's Identity

- Serialize Bob's public identity point `U_b` using `point_to_bytes`.
- Concatenate the identity and point bytes:
```
buf = ID_B || U_b
```
- Compute the hash-derived scalar:
$$h_B = H(\texttt{ID_B}||U_b) \bmod q$$
using `sha256_to_scalar`.

Shared Key Computation

- Compute the intermediate identity-bound point:
$$\texttt{temp1} = h_B \cdot U_b + D$$
using `EC_POINT_mul` and `EC_POINT_add`.
- Multiply by Alice's long-term private key:
$$\texttt{temp2} = x_a \cdot \texttt{temp1}$$
- Compute the ephemeral Diffie–Hellman component:
$$\texttt{temp1} = p_a \cdot E_b$$
- Compute the shared session key:
$$K_ab = \texttt{temp2} + \texttt{temp1}$$

Output

- Write the shared key EC point to:

  `alice_shared_key_Kab.txt`

- The shared key is an elliptic curve **point**, not a scalar.

Correctness Property

- The implementation satisfies the Arazi–Qi correctness equation:

$$K_ab = (x_a \cdot x_b + p_a \cdot p_b) \cdot P$$

- Both long-term identity keys and ephemeral keys contribute to the final shared secret.

Security Considerations

- The ephemeral scalar `p_a` must be freshly generated for each protocol run.
- Reuse of `p_a` compromises forward secrecy.
- Alice never learns Bob's private key `x_b`.

3. bob

This file implements **Bob's online phase** of the ECC-based Arazi–Qi identity-based authenticated key exchange protocol. Bob generates an ephemeral Diffie–Hellman component and independently computes the shared session key.

Inputs and Identity

- Bob's fixed identity string:

  `ID_B = "bob@example.com"`

- Alice's identity string (used for hashing):

  `ID_A = "alice@example.com"`

- Input files (hex encoded):
  - `bob_private_xb.txt`
  - `bob_public_Ub.txt`
  - `alice_public_Ua.txt`
  - `ca_master_public_D.txt`

Elliptic Curve Initialization

- Initialize the elliptic curve group and group order using:

  `init_group(&group, &q)`

- Obtain the generator point:

  $P =$ `EC_GROUP_get0_generator(group)`

- Allocate a big-number context `ctx` using `BN_CTX_new`.

Loading Long-Term Parameters

- Load Bob's long-term private scalar `x_b` from `argv[1]` using `read_bn_hex`.
- Load Bob's public identity point `U_b` from `argv[2]` using `read_point_hex`.
- Load Alice's public identity point `U_a` from `argv[4]`.
- Load the CA master public key `D` from `argv[5]`.

Ephemeral Key Generation

- Load Bob's ephemeral scalar `p_b` from `argv[3]`.
- Compute Bob's ephemeral public point:
$$E_b = p_b \cdot P$$
using `EC_POINT_mul`.
- Write the ephemeral public point to:
`bob_ephemeral_Eb.txt`
- Bob conceptually sends the tuple:
$$(\texttt{ID_B}, U_b, E_b)$$
to Alice.

Receiving Alice's Ephemeral Contribution

- Attempt to load Alice's ephemeral public point `E_a` from:
`alice_ephemeral_Ea.txt`
- If the file does not exist, Bob terminates after writing `E_b`.

Hash Computation for Alice's Identity

- Serialize Alice's public identity point `U_a` using `point_to_bytes`.
- Concatenate the identity and point bytes:
```
buf = ID_A || U_a
```
- Compute the hash-derived scalar:
$$h_A = H(\texttt{ID_A} || U_a) \bmod q$$
using `sha256_to_scalar`.

Shared Key Computation

- Compute the intermediate identity-bound point:

$$\texttt{temp1} = h_A \cdot U_a + D$$

  using `EC_POINT_mul` and `EC_POINT_add`.

- Multiply by Bob's long-term private key:

$$\texttt{temp2} = x_b \cdot \texttt{temp1}$$

- Compute the ephemeral Diffie–Hellman component:

$$\texttt{temp1} = p_b \cdot E_a$$

- Compute the shared session key:

$$K_ab = \texttt{temp2} + \texttt{temp1}$$

Output

- Write the shared key EC point to:

  `bob_shared_key_Kab.txt`

- The output must match `alice_shared_key_Kab.txt` exactly.

Correctness Property

- Bob's computation satisfies the Arazi–Qi correctness equation:

$$K_ab = (x_a \cdot x_b + p_a \cdot p_b) \cdot P$$

- Both Alice and Bob independently derive the same EC point.

Security Considerations

- The ephemeral scalar `p_b` must be freshly generated for each session.
- Bob never learns Alice's private scalar `x_a`.
- Malformed EC points must be rejected before computation.
- All arithmetic on EC points uses group operations only.

Rubric:

- Correct implementation of **ca.c** **[10 pt]**
- Correct implementation of **alice.c** **[15 pt]**
- Correct implementation of **bob.c** **[15 pt]**

Programming Assignment 2 : Schnorr HIBS [60 points]

Implementation Setting

- You will use openssl for all cryptogrpahic implementations.
- The file **RequiredFunctions.c** contains helper functions that you are allowed to use in your implementation. Make sure to read through this file carefully and understand each function, as you will need to use them in your main code. DO NOT MODIFY THIS FILE.
- All cryptographic keys, ciphertexts, and signatures exchanged via files will be Hex-encoded strings.
- You will also be provided with templates for this assignment. They also have detailed instructions on each step you should perform.
- Test Vectors will be provided to validate your outputs.
- A Verification script **VerifyHIBS.sh** will be provided to you. use the following command to run it:

```
bash VerifyHIBS.sh
```

  Your code, test vectors, required function file and this verification script must be kept in the same directory. This script will run the programs in the following order:

$$PKG\ (Level\ 0) \rightarrow Signer1\ (Level\ 1) \rightarrow Signer2\ (Level\ 2) \rightarrow Verifier$$

  PKG generates keys for Signer1. Signer1 generates keys for Signer2. Signer2 signs a message and the Verifier verifies it. The verification script will compare your results with the test vectors by comparing the file names and its contents. So make sure that files generated by your code has correct names and format.

1. pkg.c

This file implements the **system setup algorithm** for the Schnorr-based Hierarchical Identity-Based Signature (HIBS) scheme. It represents the **Level 0 Private Key Generator (PKG)**.

Cryptographic Setting

- Elliptic curve group:

$$G = \langle P \rangle$$

  of prime order `q`.
- Scalars live in the finite field:

$$\mathbb{Z}_q$$

- Elliptic curve points live in group:

$$G$$

- Master secret key:

$$x \in \mathbb{Z}_q$$

- Master public key:

$$\texttt{mpk} = x \cdot P$$

HIBS Context

- The PKG corresponds to hierarchy level 0.
- The level-0 secret key is defined as:
$$\texttt{sk_ID0} = x$$
- This key is used exclusively to derive private keys for level-1 signers.

Elliptic Curve Initialization

- Initialize the elliptic curve group and group order using:
$$\texttt{init_group(\&group, \&q)}$$
- Obtain the generator point:
$$P = \texttt{EC_GROUP_get0_generator(group)}$$
- The value `q` is extracted from the curve order.

Master Secret Key Input

- The PKG expects exactly one command-line argument:
$$\texttt{argv[1]} = \texttt{pkg_x.txt}$$
- Load the master secret key `msk` from file using:
$$\texttt{read_bn_hex(argv[1], \&msk)}$$
- The scalar `msk` satisfies:
$$1 \leq \texttt{msk} \leq q - 1$$

Master Public Key Generation

- Allocate an elliptic curve point `mpk`.
- Compute the master public key using:
$$\texttt{mpk} = \texttt{msk} \cdot P$$
via `EC_POINT_mul`.

Output

- Write the master public key to the file:
$$\texttt{mpk.txt}$$
using `write_point_hex`.
- The master secret key `msk` is **never written to disk** by this file.

Security Properties

- The master secret `msk` remains confined to the PKG.
- This file performs:
  - No signing
  - No verification
  - No message processing
- All elliptic curve operations preserve group structure:
  - EC points are never reduced modulo a prime
  - Scalars are never treated as EC points

2. signer1.c

This phase implements `HIBS.Extract` for hierarchy level $k = 1$. The Private Key Generator (PKG) delegates a private key to the Level 1 signer associated with identity `ID_1`.

- The implementation is contained in the file `signer1.c`.
- The PKG-level secret key `sk_ID0` is provided as a scalar `x`.
- Public elliptic curve parameters `group`, `P`, and `q` are shared system-wide.

Inputs

- `ID1.txt`: File containing the identity string `ID_1`
- `signer1_b1.txt`: File containing the random scalar `b1`
- `msk.txt`: File containing the PKG master secret `x`

Step-by-Step Procedure

1. Initialize the elliptic curve environment by calling `init_group(&group, &q)`.
2. Obtain the generator point `P` using `EC_GROUP_get0_generator(group)`.
3. Read the PKG master secret scalar `x` from `msk.txt` using `read_bn_hex()`. This value represents `sk_ID0`.
4. Read the identity string `ID_1` from `ID1.txt` and remove newline characters to obtain a clean identity.
5. Read the random scalar `b1` from `signer1_b1.txt`. The value `b1` is sampled uniformly from $\mathbb{Z}_q$.
6. Compute the public identity point
$$Q_ID1 = b1 \cdot P$$
using `EC_POINT_mul()`.
7. Serialize the elliptic curve point `Q_ID1` into a byte array using `point_to_bytes()`.
8. Concatenate the identity string `ID_1` with the serialized representation of `Q_ID1`.

9. Compute the hash scalar
$$c_ID1 = H1(ID_1 \,\|\, Q_ID1)$$
using `H1_to_scalar()`, ensuring the result lies in $\mathbb{Z}_q$.

10. Compute the Level 1 private key
$$sk_ID1 = x \cdot c_ID1 + b1 \pmod{q}$$
using `BN_mod_mul()` followed by `BN_mod_add()`.

11. Write the Level 1 private key `sk_ID1` to `sk_ID1.txt` using `write_bn_hex()`.

12. Write the public identity point `Q_ID1` to `Q_ID1.txt` using `write_point_hex()`.

Correctness Invariant

The implementation satisfies the Schnorr-HIBS delegation invariant:

$$sk_ID1 \cdot P = c_ID1 \cdot mpk + Q_ID1$$

where `mpk` is the master public key generated by the PKG.

Security Properties

- The master secret `x` is never revealed as an elliptic curve point.
- The signer learns only `sk_ID1`, not the PKG master secret.
- The PKG's influence is cryptographically bound to all derived keys.
- The signer at Level 1 may further delegate keys but cannot remove PKG control.

3. Signer2.c

This phase implements the `HIBS.Extract` algorithm for hierarchy level $k = 2$, as well as the Schnorr signature generation algorithm. The Level 2 signer derives its private key from a Level 1 parent key and produces signatures on messages.

Role of signer2.c

- Performs hierarchical key delegation from Level 1 signer to Level 2 signer.
- Implements Schnorr signature generation for messages.
- Uses parent key (`sk_ID1`) and public point (`Q_ID1`) from Level 1 to derive `sk_ID2`.

Inputs

- `sk_ID1.txt`: Level 1 private key scalar
- `Q_ID1.txt`: Level 1 public point
- `signer2_b2.txt`: Level 2 random delegation scalar
- `signer2_r.txt`: Random scalar for Schnorr signing
- `ID2.txt`: Identity string for Level 2 signer
- `message.txt`: Message to sign

Level 2 Delegation Procedure

1. Read the Level 1 parent key `sk_ID1` and public point `Q_ID1`.
2. Read the identity string `ID_2` and random scalar `b2`.
3. Compute the Level 2 public point:
$$Q_ID2 = b2 \cdot P$$
using `EC_POINT_mul()`.
4. Compute the hash-derived scalar for delegation:
$$c_ID2 = H1(ID_2 \,\|\, Q_ID1 \,\|\, Q_ID2)$$
using `H1_to_scalar()`.
5. Compute the Level 2 private key:
$$sk_ID2 = sk_ID1 \cdot c_ID2 + b2 \bmod q$$
using `BN_mod_mul()` and `BN_mod_add()`.
6. Store the Level 2 public point `Q_ID2` in `Q_ID2.txt`.

Schnorr Signing Procedure

1. Read the message `m` and signing nonce `r`.
2. Compute:
$$R = r \cdot P$$
using `EC_POINT_mul()`.
3. Compute the challenge hash:
$$h = H2(m \,\|\, R)$$
using `H2_to_scalar()`.
4. Compute the Schnorr signature:
$$s = r + h \cdot sk_ID2 \bmod q$$
using `BN_mod_mul()` and `BN_mod_add()`.
5. Output the signature pair (s, h) to `sig_s.txt` and `sig_h.txt`.

Correctness Invariants

- Level 2 delegation satisfies:
$$sk_ID2 \cdot P = c_ID2 \cdot Q_ID1 + Q_ID2$$
- Schnorr signature satisfies:
$$s \cdot P = R + h \cdot (sk_ID2 \cdot P)$$
ensuring verification correctness.

Security Considerations

- The random value `b2` ensures unlinkability between Level 2 keys.
- The nonce `r` must be fresh for every signature to maintain Schnorr security.
- `sk_ID2` must never be exposed.
- The Level 2 signer cannot bypass the hierarchical influence of Level 1 and PKG.

verifier.c

This phase implements the `HIBS.Verify` algorithm for a 2-level hierarchical Schnorr signature. The verifier checks the validity of a signature using only public information.

Role of verifier.c

- Accepts a message and its signature (s, h).
- Uses public parameters and delegation points from Level 1 and Level 2.
- Reconstructs the effective public key and validates the signature.
- Does not require any private keys.

Inputs

- `ID1.txt`, `ID2.txt`: Identity strings for Level 1 and Level 2.
- `message.txt`: Message to verify.
- `mpk.txt`: Master public key.
- `Q_ID1.txt`, `Q_ID2.txt`: Delegation points.
- `sig_s.txt`, `sig_h.txt`: Signature components.

Verification Procedure

1. Compute the hash-derived scalars:

$$c_ID1 = H1(ID_1 \,\|\, Q_ID1), \quad c_ID2 = H1(ID_2 \,\|\, Q_ID1 \,\|\, Q_ID2)$$

2. Reconstruct the effective public key:

$$PK_eff = (c_ID1 c_ID2) \cdot mpk + (c_ID2) \cdot Q_ID1 + Q_ID2$$

3. Compute the reconstructed commitment point:

$$R' = s \cdot P - h \cdot PK_eff$$

4. Compute the hash of the message and reconstructed point:

$$h' = H2(m \,\|\, R')$$

5. Accept the signature if and only if:

$$h' = h$$

Correctness Guarantee

- If the signature was generated honestly:
$$R' = r \cdot P$$
- The signature verification equation holds:
$$s \cdot P = R' + h \cdot PK_eff$$

Security Considerations

- Verification is stateless and does not require contacting the PKG.
- The verifier trusts no party; only public parameters are used.
- All scalars are reduced modulo q, and invalid EC points are rejected.
- The scheme ensures hierarchical authenticity: signatures can be traced to specific identities through delegation points.

Implementation Notes

- Use `EC_POINT_mul` for scalar-point multiplications.
- Use `EC_POINT_add` and `EC_POINT_invert` for point arithmetic.
- Serialize EC points before hashing.
- Store intermediate values carefully to avoid memory leaks.

Rubric:

- Correct implementation of **pkg.c** **[15 pt]**
- Correct implementation of **signer1.c** **[15 pt]**
- Correct implementation of **signer2.c** **[15 pt]**
- Correct implementation of **verifier.c** **[15 pt]**